Precision Engineering and Kernel Optimization for Maximum Token Throughput Across Google Colab Accelerators

Executive Audit of the Provided Precision Specifications

An architectural audit of the six reference documents covering Google Colab's hardware accelerators—TPU v6e (Trillium), NVIDIA G4 (RTX PRO 6000 Blackwell Server Edition), NVIDIA A100, NVIDIA L4, NVIDIA Tesla T4, and TPU v5e—confirms that their foundational premise is sound[1]. Autoregressive large language model (LLM) inference is governed by two distinct hardware regimes: the prefill and high-concurrency batching phase, which is bottlenecked by matrix multiplication compute density (TFLOPS or TOPS), and the low-batch token generation (decode) phase, which is strictly bottlenecked by off-chip memory bandwidth (HBM or GDDR transfer rates)[1]. Because generating a single token at batch size $B = 1$ requires streaming every active parameter from memory into the execution registers while performing minimal arithmetic per byte fetched, shrinking the byte representation of weights and the Key-Value (KV) cache yields near-linear speedups in single-stream decoding[1].

However, cross-referencing the six documents against low-level compiler documentation, hardware specifications, and kernel execution benchmarks reveals several critical inaccuracies, mathematical inversions, and silicon-level nuances that must be corrected to achieve peak tokens per second in production notebooks.

Technical Corrections and Silicon Nuances by Accelerator

The TPU v6e reference document accurately identifies that a single Trillium chip features 32 GB of High-Bandwidth Memory (HBM) operating at 1,638 to 1,640 GB/s—doubling the 16 GB capacity and 819 GB/s bandwidth of the TPU v5e—and expands the systolic Matrix Multiply Unit (MXU) from 128×128 to 256×256 [1]. However, the document claims that TPU v6e executes native FP8 tensor math at 1,836 TFLOPs per chip, doubling the 918 TFLOPs of native bfloat16 (BF16)[1]. Official Google Cloud hardware specifications indicate that on TPU v6e, the 1,836 TOPs (2× peak compute) hardware rating applies specifically to **INT8** integer arithmetic, whereas peak **FP8** compute on v6e is rated at 918 TFLOPs, which is identical to BF16[8]. Native 2× FP8 compute scaling over BF16 (4,614 TFLOPs FP8 vs. 2,307 TFLOPs BF16) arrives with the subsequent TPU v7x (Ironwood) architecture[9]. Consequently, while running FP8 on TPU v6e cuts HBM weight and KV cache traffic by 50% and delivers the stated 1.8× to 2× decode speedup in memory-bound regimes[1], compute-bound prefill and high-batch serving on v6e require **W8A8 INT8** (via Accurate Quantized Training/Inference, AQT, or compressed-tensors)

to unlock the full 1,836 TOPs systolic array ceiling[8]. Additionally, the document states the decode latency formula as HBM Memory Bandwidth / (Model Weights Size + KV Cache Size)[1]; dimensionally, dividing bytes per second by bytes yields inverse latency (tokens per second), whereas true latency per token is bounded by the ratio of total memory footprint to available bandwidth[15]:

$$t_{\text{decode}} \approx \frac{M_{\text{weights}} + M_{\text{KV}}}{B_{\text{HBM}}}$$

A second discrepancy arises from a direct contradiction between the TPU v6e and TPU v5e documents regarding 4-bit weight quantization (W4A16). The TPU v6e document advises against W4A16 (INT4 weights, BF16 activations), arguing that dynamic dequantization on the Vector Processing Unit (VPU) degrades memory-bandwidth gains[1]. Conversely, the TPU v5e document recommends W4A16 as the winning format at $B = 1$, noting that saving 75% of HBM transfer time dwarfs the VPU unpacking cost[6]. Evaluating the roofline operational intensity resolves this contradiction: TPU v5e has a ridge point of $\frac{197\ \text{TFLOPs}}{819\ \text{GB/s}} \approx 240.5\ \text{FLOPs/byte}$[6], whereas TPU v6e has a ridge point of $\frac{918\ \text{TFLOPs}}{1{,}638\ \text{GB/s}} \approx 560.4\ \text{FLOPs/byte}$[8]. Because v6e possesses 4.66× the compute of v5e but only 2× the memory bandwidth, v6e is even more severely memory-bandwidth starved at $B = 1$ than v5e[16]. Thus, W4A16 (compressed-tensors W4A16 on gmm_v2 or AWQ in vllm-jax) is mathematically superior for single-stream decoding on both TPUs, provided the runtime uses fused grouped matrix multiplication (gmm_v2) or Pallas kernels rather than allowing Hugging Face Transformers to eagerly unpack 4-bit weights into persistent BF16 buffers on the first forward pass[12].

On the NVIDIA side, the G4 document treats NVIDIA Blackwell as a unified execution target, recommending NVFP4 (W4A4) with an FP8 or FP4 KV cache for maximum throughput[2]. While the RTX PRO 6000 Blackwell Server Edition provides 96 GB of GDDR7 ECC memory at 1,597 GB/s and 752 fifth-generation Tensor Cores rated at 4 PFLOPS of peak FP4 compute, 2 PFLOPS of FP8, and 1 PFLOPS of BF16/FP16[19], it is built on the workstation/prosumer Blackwell silicon (**Compute Capability SM 12.0 / sm_120**) rather than datacenter B200/B300 silicon (**Compute Capability SM 10.0 / sm_100**)[22]. Pre-compiled sm_100 binaries crash on sm_120, and unpatched CUTLASS/FlashInfer autotuners on SM120 fall back to non-TMA (Tensor Memory Accelerator) tactics on NVFP4 Mixture-of-Experts (MoE) layers, running at 14.6 tokens/sec—slower than 4-bit Marlin weight-only kernels (46 to 49 tokens/sec)—unless compiled with CUDA 13.0+ targeting the compute_120f architecture suffix[22]. Furthermore, on SM100+ and SM120 targets, legacy **INT8** GEMM dispatch paths are excluded from compilation (gated to 7.5;8.0;8.7;8.9+PTX), meaning INT8 models silently fail to accelerate on Blackwell whereas FP8 executes natively[24].

Regarding the NVIDIA A100 (sm_80), the reference document recommends setting kv_cache_dtype="fp8_e5m2" in vLLM to compress the KV cache to 8-bit on Ampere[3]. While

SM80 cannot compile fp8_e4m3 (fp8e4nv) in Triton and relies on fp8_e5m2 for 8-bit storage[13], empirical testing reveals three critical caveats. First, certain modern model definitions in vLLM (such as Gemma 4) contain hard assertions (assert kv_cache_dtype in {"fp8", "fp8_e4m3"}) that reject fp8_e5m2 outright, forcing Ampere back to BF16/FP16 KV caches[13]. Second, under full CUDA graph capture on SM80, unpatched _insert_kv paths for fp8_e5m2 silently corrupt attention outputs into garbage unless eager breaks or piecewise graphs are used[25]. Third, because Ampere lacks native FP8 attention hardware and upconverts KV entries to 16-bit in registers, fp8_e5m2 on A100 increases maximum concurrency and context length (~1M tokens) but reduces raw single-stream decode speed to approximately 90% of the BF16 baseline[25]. Finally, both the A100 and T4 documents state that FP8 checkpoints must be avoided on these GPUs because native FP8 Tensor Cores require Ada Lovelace (sm_89) or Hopper (sm_90)[3]. While this holds true for native W8A8 matrix math, vLLM includes a dedicated **FP8-Marlin (MarlinFP8ScaledMMLinearKernel)** backend that supports Compute Capability ≥ 7.5, covering both the Turing T4 and Ampere A100[27]. This kernel executes FP8 checkpoints as weight-only **W8A16**, streaming 1-byte FP8 weights from memory and dequantizing them inside registers to FP16/BF16 prior to Tensor Core execution[28]. Similarly, Ampere (sm_80) and Ada (sm_89) can execute **NVFP4** checkpoints as weight-only W4A16 via Marlin in-kernel dequantization when enabled via runtime flags[30].

Accelerator (Colab Runtime)	**Architecture & Compute Cap.**	**Memory Capacity & Bandwidth**	**Peak 16-Bit Compute**	**Peak 8-Bit Compute**	**Peak 4-Bit Compute**	**Optimal Low-Batch Decode (B=1–4 B=1\text{--}4B=1–4)**	**Optimal High-Batch Serving (B≥16B \ge 16B≥16)**
NVIDIA G4 (RTX PRO 6000)	Blackwell (sm_120 / 120f)[2]	96 GB GDDR7 ECC @ 1,597 GB/s[19]	1,000 TFLOPS (BF16/FP16)[20]	2,000 TFLOPS (FP8 Native)[20]	4,000 TFLOPS (NVFP4/MXFP4)[21]	**NVFP4 / MXFP4 (W4A4)** or **W4A16 Marlin** + FP8/NVFP4 KV[2]	**NVFP4 (W4A4)** or **FP8 (W8A8)** + FP8 KV[2]

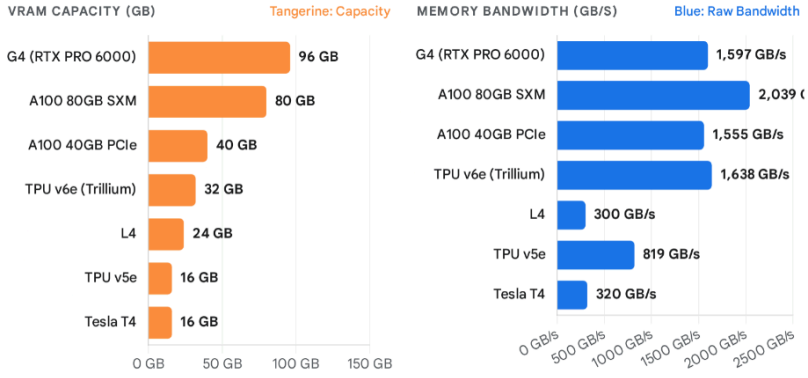
NVIDIA A100 (80GB SXM / 40GB)	Ampere (sm_80)[3]	80 GB @ 2,039 GB/s / 40 GB @ 1,555 GB/s[3]	312 TFLOPS (BF16/FP16)[3]	624 TOPS (INT8 Native; No Native FP8)[3]	No Native FP4; INT4 Tensor Core[3]	**W4A16 (AWQ/ GPTQ Marlin)** + FP16/BF16 KV[3]	**W8A8 SmoothQuant INT8** or **W4A16 Marlin**[3]
Google TPU v6e (Trillium)	TPU v6e (1 Tensor Core, $256 \times$ MXU)[8]	32 GB HBM @ 1,638–1,640 GB/s[7]	918 TFLOPS (BF16 Native; Avoid FP16)[1]	1,836 TOPs (INT8) / 918 TFLOPS (FP8)[8]	No Native 4-Bit MXU (VPU Unpack)[1]	**W4A16 (gmm_v2 / AWQ)** or **FP8 (W8A8)** + FP8 KV[1]	**W8A8 INT8 (AQT)** or **W8A8 FP8** + FP8 KV[1]
NVIDIA L4	Ada Lovelace (sm_89)[4]	24 GB GDDR6 @ 300 GB/s[4]	121 TFLOPS Dense (BF16/FP16)[4]	242 TFLOPS Dense (FP8 Native)[4]	No Native FP4; INT4 via Marlin[4]	**W4A16 (AWQ/ GPTQ Marlin)** + FP8/Auto KV[4]	**FP8 (W8A8)** + FP8 KV Cache[4]
Google TPU v5e	TPU v5e (1 Tensor Core, $128 \times$ MXU)[6]	16 GB HBM @ 819 GB/s[6]	197 TFLOPS (BF16 Native; Avoid FP16)[6]	393 TOPs (INT8 Native; No FP8)[6]	No Native 4-Bit MXU (VPU Unpack)[6]	**W4A16 (INT4 + BF16)** + INT8 Ring KV Cache[6]	**W8A8 INT8 (AQT / MaxText)** + INT8 KV Cache[6]
NVIDIA Tesla T4	Turing (sm_75)[5]	16 GB GDDR6 @ 300–320 GB/s[5]	65 TFLOPS (FP16 Only; No BF16)[5]	130 TOPS (INT8; FP8 via W8A16 Marlin)[5]	260 TOPS (INT4 Native; EXL2/Marlin)[5]	**INT4 (EXL2 / AWQ-Marlin / GGUF)** + FP16/Q	**W4A16 AWQ-Marlin** or **INT8 (W8A8)**[5]

						8 KV[5]	

Memory Bandwidth Dictates Single-Stream Decode While Sub-Byte Compute Rules High-Batch Serving

Panel 1: Memory Wall Baseline (Capacity & Bandwidth)

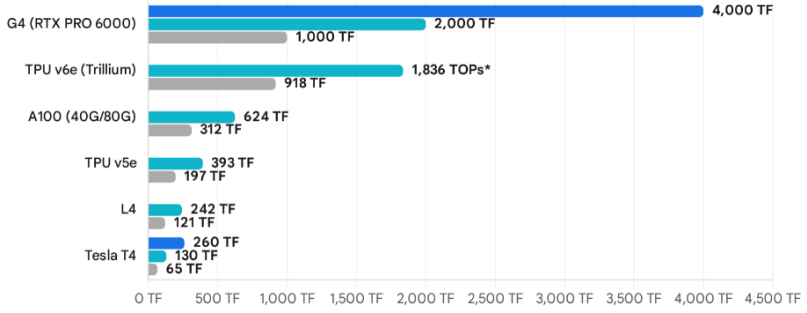
Batch=1 speed ceiling is bound by GB/s; model footprint by GB



Panel 2: Peak Compute Scaling Across Precision Formats

4-bit (NVFP4/INT4) · 8-bit (FP8/INT8) · 16-bit (BF16/FP16)

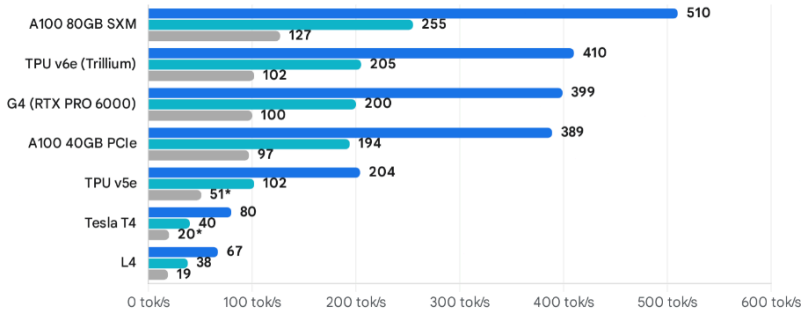
Architecture Note TPU v6e delivers **1,836 TOPs in INT8** (FP8 is 918 TFLOPs). G4 RTX PRO 6000 reaches **4,000 TFLOPS** with Blackwell NVFP4 Tensor Cores.



Panel 3: Theoretical 8B Model Single-Stream Decode Ceiling (Tokens/sec, Batch=1)

4-bit (~4 GB Read) · 8-bit (~8 GB Read) · 16-bit (~16 GB Read)

VRAM Ceiling **OOM Alert:** 16 GB devices (Tesla T4, TPU v5e) cannot hold an 8B model + KV cache in 16-bit without Out-of-Memory. Quantization (8-bit / 4-bit) is mandatory.

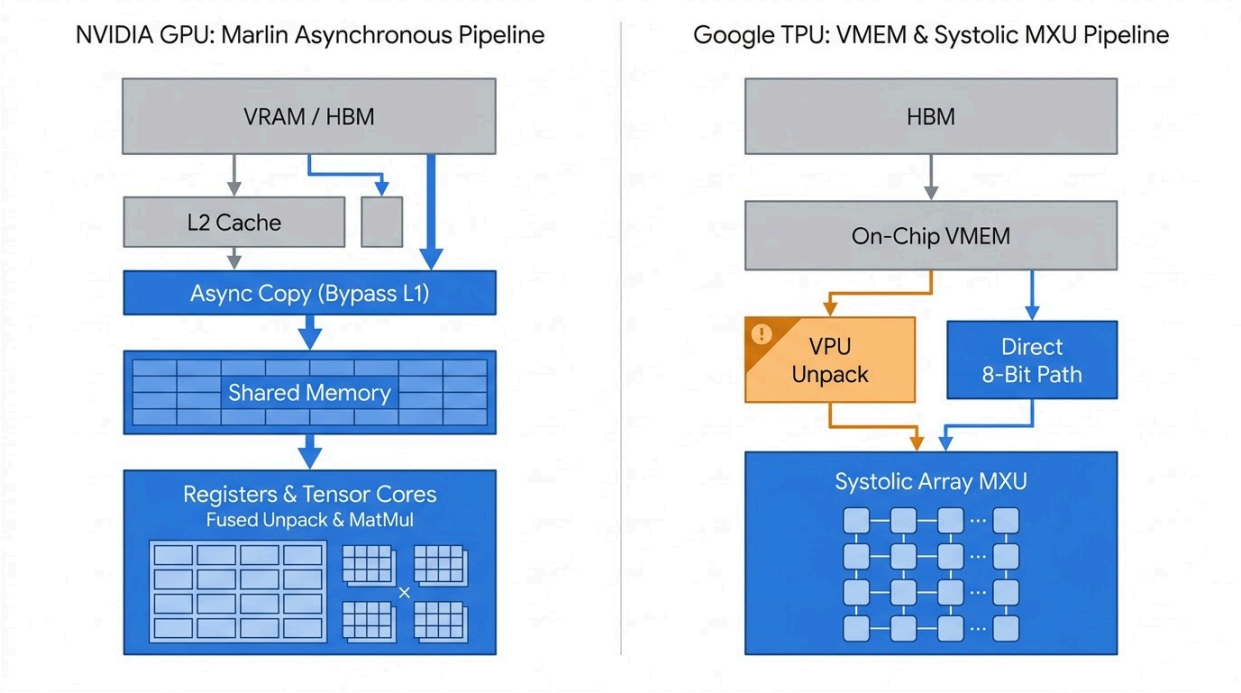


Hardware capabilities and theoretical single-stream decode limits across Google Colab accelerators. While the **NVIDIA G4 (RTX PRO 6000 Blackwell)** delivers up to 4,000 TFLOPS in NVFP4 compute and 96 GB of capacity, single-stream decode speed at batch size 1 tracks raw memory bandwidth, allowing the **A100 80GB** (2,039 GB/s) and **TPU v6e** (1,638 GB/s) to rival or exceed G4 (1,597 GB/s) per byte transferred.

Data sources: Google Cloud TPU Documentation, Inferbase GPU Catalog, Akash Network Hardware Catalog, Google Cloud GPU Architecture

Microarchitectural Deep Dive and Advanced Kernel Tricks by Accelerator

Achieving the theoretical token generation ceilings requires aligning the quantization format with the underlying register file, shared memory hierarchy, and matrix execution units of each accelerator. In standard inference pipelines, loading a quantized checkpoint often fails to deliver the expected speedup because naive dequantization kernels introduce memory round-trips, cache thrashing, or vector unit stalls[15].



NVIDIA G4 (RTX PRO 6000 Blackwell Server Edition): Navigating the SM120 Frontier

The Google Cloud G4 instance equips the Colab runtime with an NVIDIA RTX PRO 6000 Blackwell Server Edition GPU featuring 96 GB of GDDR7 ECC memory across a 512-bit bus, 1,597 GB/s of memory bandwidth, and a PCIe Gen 5 x16 host interface[19]. While the provided documentation rightly identifies **NVFP4** (4-bit floating point with microscaling) and **FP8 (E4M3)** as the premier formats for this card[2], unlocking its 4 PFLOPS FP4 and 2 PFLOPS FP8 Tensor Core ceiling requires overcoming several software-stack traps specific to its **SM 12.0 (sm_120)** compute capability[21]. Despite sharing the Blackwell brand with the datacenter B200 (sm_100), the RTX PRO 6000 reports device capability 12.0 (sm_120)[22]. Standard CUTLASS templates compiled with compute_120a fail to initialize Tensor Memory Accelerator (TMA) Warp-Specialized (WS) grouped GEMM tactics on SM120, causing the FlashInfer autotuner to fall back to slow non-TMA kernels—yielding only 14.6 tokens/sec on NVFP4 MoE models—or to produce corrupted output due to missing Grid Dependency Control (griddepcontrol.wait/launch) barriers[22]. Compiling with CUDA 13.0+ and the **compute_120f** ("full" feature set) target flag unlocks fast TMA WS grouped GEMM tactics on SM120, nearly tripling native FP4 MoE

generation speed to 39.0+ tokens/sec[22]. In Colab, practitioners must install the CUDA 13 FlashInfer wheel (pip install flashinfer-python[cu13]) alongside pre-compiled cubins (flashinfer install-cubin-wheel) to ensure these SM120 paths are active[39].

To maximize execution efficiency across different layer types on SM120, a hybrid backend routing strategy is required. Setting the environment variable VLLM_NVFP4_GEMM_BACKEND=cutlass directs dense linear projections to utilize vLLM's native CUTLASS kernels[22], while setting VLLM_USE_FLASHINFER_MOE_FP4=1 exclusively when serving Mixture-of-Experts architectures activates FlashInfer's fused FP4 MoE routing[40]. Furthermore, when running an fp8 or nvfp4 quantized KV cache on SM120 using FlashInfer's XQA decode path, full CUDA graph capture (FULL) silently corrupts attention outputs[32]. Passing --compilation-config '{"cudagraph_mode":"PIECEWISE"}' in vLLM isolates the attention decode step from full-graph capture corruption while preserving graph acceleration across all linear layers, enabling a 244,928-token KV cache ceiling (compared to approximately 163,000 tokens in FP8) at 95.6 tokens/sec single-stream on 27B hybrid architectures[32].

Beyond NVIDIA's proprietary nvfp4 format (generated via modelopt), recent FlashInfer releases for SM120 (b12x) introduce native **MXFP4** (Open Compute Microscaling 4-bit float) support across W4A4 fused MoE, W4A8/W4A16, and dense GEMMs[31]. These releases also incorporate Tensor Memory (TMEM) overlap for NVFP4 SwiGLU epilogues and QKV projections, as well as asymmetric VO-split NVFP4 paged prefill (qk=512/vo=256) for architectures such as Gemma 4[31]. Finally, because the RTX PRO 6000 Server Edition supports Universal Multi-Instance GPU (MIG), a single 96 GB Colab G4 card can be partitioned into up to four isolated 24 GB hardware instances (4x 24GB MIG), each with dedicated GDDR7 memory controllers, L2 cache, and Tensor Cores[19]. When running batch evaluations of 7B to 14B models in FP8 or NVFP4, deploying four parallel data-parallel workers across four MIG slices eliminates multi-stream scheduling contention.

NVIDIA A100 (40GB / 80GB) and L4: Exploiting the Marlin Pipeline and FlashInfer

On the NVIDIA A100 (sm_80, 1,555 to 2,039 GB/s HBM2e) and NVIDIA L4 (sm_89, 300 GB/s GDDR6), the primary determinant of low-batch decode speed is not merely choosing 4-bit weights (W4A16), but specifically routing execution through the **Marlin** kernel (awq_marlin or gptq_marlin)[3]. Standard GPTQ and AWQ inference kernels suffer from synchronous memory loads, random access patterns that yield poor L2 cache hit rates (30–50%), L1 cache pollution from streaming weights, and un-fused dequantization in the critical path[33]. In empirical benchmarks on Qwen2.5-32B-Instruct, switching from standard AWQ to Marlin-AWQ increases throughput from 68 tokens/sec to **741 tokens/sec** (a 10.9× speedup), while Marlin-GPTQ jumps from 276 tokens/sec to **712 tokens/sec** (a 2.6× speedup)[33]. Marlin achieves near-100% memory bandwidth utilization through four coordinated hardware mechanisms: it uses asynchronous copy instructions (cp.async) to stream packed 4-bit weights from DRAM through the L2 cache directly into Shared Memory (SMEM), bypassing the L1 cache entirely so L1 remains dedicated to activations; it orders weight tiles for spatial and temporal locality, raising L2 cache hit rates to 80–95%; it implements SMEM double buffering so tile $N + 1$ loads

asynchronously in the background while Tensor Cores compute on tile N ; and it fuses dequantization with computation by unpacking 4-bit weights to 16-bit inside the register file immediately prior to warp-level Tensor Core matrix-multiply-accumulate (MMA) instructions, eliminating intermediate global memory writes[33].

This asynchronous register-dequantization architecture also enables cross-format emulation on older Tensor Cores. Although the A100 lacks native FP8 and FP4 hardware[3], it can execute modern FP8 checkpoints at high speed using **FP8-Marlin (W8A16)**, which packs weights in 1-byte FP8 in HBM and dequantizes them in-register to FP16/BF16[27]. Similarly, setting NIM_ALLOW_NVFP4_EMULATION=1 on vLLM or SGLang allows Ampere (sm_80) and Ada (sm_89) GPUs to run Blackwell NVFP4 checkpoints by dequantizing FP4 weights to BF16 in-kernel via Marlin[30]. Conversely, when running native W8A8 FP8 on the L4 at batch sizes $B \geq 8$, practitioners should select FP8 checkpoints calibrated with **static per-tensor activation scales** over dynamic per-tensor activation scales[4]. Dynamic scaling computes the absolute maximum (amax) of activation tensors on every forward pass, introducing reduction kernel overhead that degrades latency on the L4's modest SM count, whereas static scales fuse the scaling factor directly into the CUTLASS FP8 GEMM epilogue[43].

Both the A100 and L4 further benefit from attention-layer and sparsity optimizations. Standard PagedAttention implementations copy non-contiguous KV cache pages into a contiguous temporary buffer before launching the attention kernel[40]. Setting VLLM_ATTENTION_BACKEND=FLASHINFER (or --attention-backend FLASHINFER) activates BatchPrefillWithPagedKVCacheWrapper and BatchDecodeWithPagedKVCacheWrapper, which operate directly on non-contiguous paged KV blocks and deliver a **28–30% speedup** on long-context (32K+) workloads[40]. Pre-installing the flashinfer-cubin and flashinfer-jit-cache wheels in Colab eliminates the 1-to-5 request JIT compilation warm-up penalty[39]. Additionally, Ampere (sm_80) and Ada Lovelace (sm_89) Tensor Cores include hardware Sparse Tensor Core units that double raw compute throughput (from 312 to 624 TFLOPS on A100, and 242 to 484 TFLOPS FP8 on L4) when weights follow a 2:4 semi-structured sparsity pattern (two non-zero values per four-element block)[3]. In vLLM, compressed_tensors_24 schemes and sparse TensorRT-LLM engines exploit this hardware path to accelerate compute-bound prefill and high-batch generation[41].

NVIDIA Tesla T4 (Turing SM75): Squeezing Maximum Bandwidth from Legacy Silicon

The Tesla T4 (sm_75, 16 GB GDDR6, ~300–320 GB/s bandwidth) lacks bfloat16 and native fp8 hardware, making float16 and integer quantization (INT8/INT4) mandatory[5]. Beyond the standard recommendation of AWQ, GPTQ, or EXL2[5], three specific kernel and quantization optimizations unlock substantially higher throughput on T4 Colab instances. First, vLLM’s minimum compute capability gate for awq_marlin, gptq_marlin, and fp8 (weight-only W8A16 via FP8-Marlin) is 75 (sm_75)[27]. While native FP8 W8A8 fails on the T4, loading an FP8 or AWQ model in vLLM on a T4 utilizes the Marlin weight-only dequantization path to stream 8-bit or 4-bit weights over the 320 GB/s bus and execute math on Turing's FP16 Tensor Cores[28].

Second, because the T4 has a strict 16 GB VRAM ceiling, standard 4.0-bit quantization struggles to fit 14B to 32B models alongside a usable KV cache[5]. ExLlamaV2's EXL2 format supports mixed fractional bitrates across layers and heads (such as 3.0 to 3.75 bits per weight average, assigning 4 to 6 bits to sensitive attention projections and 2 to 3 bits to MLP down-projections), enabling a 27B or 32B model to fit entirely within 16 GB VRAM while decoding at 25 to 40 tokens/sec via custom register-fused CUDA kernels[5]. Third, when deploying GGUF models on a T4, compiling llama.cpp with -DGGML_CUDA_FORCE_MMQ=ON forces Matrix Multiply Quantized (MMQ) kernels that utilize Turing's native 260 TOPS INT4/INT8 dot-product instructions rather than dequantizing to FP16 for cuBLAS[5]. Pairing this build flag with -ctk q8_0 -ctv q8_0 (or q4_0) and -fa (FlashAttention) quantizes the KV cache to 8-bit or 4-bit integers, halving memory bandwidth overhead at context lengths above 2,048 tokens[5].

Google Cloud TPU v6e (Trillium) and TPU v5e: XLA Lowering, TorchTPU, and Pallas Kernels

Running LLMs on Google Colab's TPU v5e (16 GB HBM, 819 GB/s) and TPU v6e (32 GB HBM, 1,638 GB/s) requires a fundamentally different optimization paradigm than CUDA GPUs because TPUs rely on ahead-of-time and just-in-time whole-graph compilation via OpenXLA rather than dynamic per-operator kernel launches[8]. The TPU software stack has transitioned away from legacy Lazy Tensor torch_xla toward two high-performance pathways: **TorchTPU (torch_tpu)** for native PyTorch (model.to("tpu") and torch.compile(model, backend="tpu"))[47], and **vllm-jax (formerly tpu-inference)** for unified JAX/PyTorch serving via vLLM[50]. On a single Colab TPU v5e-1 chip, running PyTorch in strict eager mode yields only **5.0 tokens/sec**, and TORCH_TPU_DEFER_AND_FUSE=1 reaches only **9.1 tokens/sec** while suffering a 359-second initial compilation stall because autoregressive generation grows sequence length by 1 token per step, triggering a fresh XLA compilation for every shape[18]. Wrapping the decode step in torch.compile(backend="tpu") with **Bounded Dynamism** and a fixed-size **Ring KV Cache** eliminates shape recompilations and boosts single-chip v5e-1 decode speed to **45.3 to 50.0 tokens/sec**—a 10× speedup over eager mode—while holding a 131,328-token context window in just 0.82 GB of HBM[18].

Even after graph compilation, a hidden bottleneck on Colab TPU runtimes is token-by-token host synchronization. If a Python generation loop reads each newly sampled token ID back from the TPU to the CPU host on every step for live text streaming, PCIe round-trip latency artificially caps TPU v5e-1 throughput at **31.8 to 37.2 tokens/sec**, completely erasing the speed advantage of 4-bit quantization[18]. Accumulating generated token IDs inside an on-chip HBM buffer across the generation window and reading them back to the host in multi-token chunks or at End-of-Sequence (EOS) immediately restores throughput to 45–50+ tokens/sec[18].

Maintaining true sub-byte weight compression in HBM is equally critical on TPUs. Standard Hugging Face compressed-tensors integrations unpack every 4-bit layer to bfloat16 during the first forward pass and retain the unpacked BF16 weights in HBM, negating both memory savings and bandwidth speedups[18]. To execute true W4A16 on TPUs, vllm-jax provides the **compressed-tensors W4A16 on gmm_v2** linear method, which retains 4-bit packed weights in

HBM and unpacks tiles inside Vector Memory (VMEM) during the grouped matrix multiply[17]. Furthermore, for models with large vocabularies (such as Gemma), keeping the embedding table quantized on a 4-bit integer grid with a per-row bfloat16 scale factor (self.embed_scale) frees **3.4 GB of HBM** on a v5e-1 chip (dropping the loaded footprint from 9.82 GB to 6.44 GB, which is vital when XLA compilation peak memory reaches 16.67 GB against the chip's 16 GiB / 17.2 GB physical limit) and accelerates the compiled decode step by **1.51× to 1.60×**[18].

When implementing blockwise (subchannel) FP8 or INT4 quantization via QWIX, Tokamax (use_tokamax_gmm=True), or custom Pallas kernels on TPU v6e (or v7), tensor block dimensions **must be multiples of 256** to match the expanded 256×256 systolic array[9]. Empirical profiling demonstrates that subchannel block sizes smaller than 384 (such as 1×128) starve the MXU and make the kernel **VPU-bound** due to per-block scale multiplication overhead, causing quantized dot_general operations to run slower than unquantized BF16[36]. Using per-channel scaling or block sizes of 1×256 or 1×512 written directly inside Pallas kernels restores the full ~2× hardware speedup[36]. Finally, in vllm-jax on TPU v6e and v5e, passing --attention-backend to select **Ragged Paged Attention V3 (Batched or Stacked RPA v3)** stops over-fetching KV pages during decode and removes mixed-batch padding penalties during speculative verification[12]. Because TPU v6e also integrates **two SparseCores** per chip[8], pinning embedding gather operations and collective communications to a dedicated SparseCore stream (torch.tpu.Stream(pinned="sparse")) in TorchTPU and XLA overlaps communication and sparse lookups asynchronously with MXU matrix multiplications[47].

Cross-Platform Algorithmic and Runtime Accelerators for Google Colab

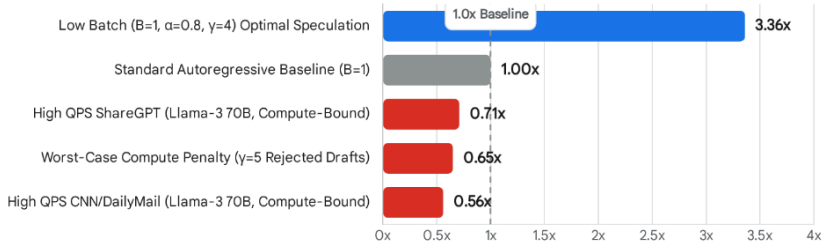
Beyond hardware-specific precision formats and low-level kernels, engine-level algorithmic transformations can multiply effective token generation speed across all six accelerators without sacrificing output accuracy.

Kernel Selection, Graph Compilation, and Low-Batch Speculation Multiply Raw Precision Gains

LEGEND: Optimal / Compiled Peak · Accelerated Alternative · Host-Bottlenecked · Baseline · Degradation / Fallback Trap

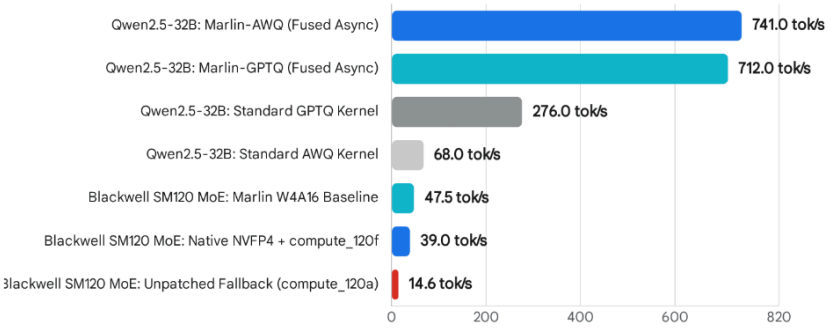
Panel A: Speculative Decoding Speedup vs. High-QPS Saturated Verifier

Tokens per Step Multiplier (Target Step = 1.0x)



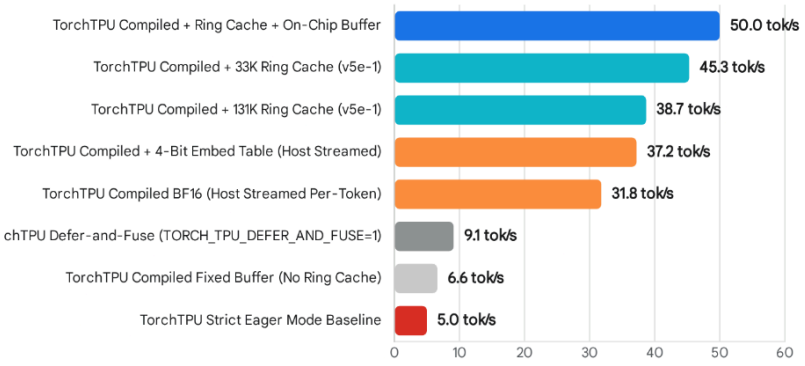
Panel B: GPU Kernel Speedup — Fused Marlin & SM120 CUDA 13 Compilation

Decode Throughput (tokens / second)



Panel C: Google Colab TPU v5e-1 — TorchTPU Compilation Progression

Single v5e-1 Chip Decode Speed (tokens / second)



Empirical performance impact of runtime and kernel optimizations. **Panel A** illustrates how speculative decoding yields up to 3.36 tokens per step at low concurrency but degrades throughput by up to 44% (1.8x slowdown) when high QPS saturates compute. **Panel B** shows the 10.9x throughput jump from enabling Marlin kernels on Qwen2.5-32B and the 2.67x gain from CUDA 13 *compute_120f* on Blackwell SM120. **Panel C** demonstrates the 10x decode acceleration on a Colab TPU v5e-1 chip achieved by combining TorchTPU compilation, on-chip token buffering, and a static ring cache.

Data sources: Dev.to (Speculative Decoding Acceptance Math), vLLM Blog, JarvisLabs, r/LocalLLaMA, Dev.to (PyTorch on TPU Colab)

Speculative Decoding: Turning Idle Compute into Memory Bandwidth Multiplication

During single-stream or small-batch autoregressive decoding ($B = 1 \text{ to } 4$), generating a token performs roughly 1 to 2 FLOPs per byte of weight data read from memory[51]. Because modern accelerators have arithmetic intensity ridge points between 200 and 2,500 FLOPs/byte, the Tensor Cores or MXUs sit idle for the vast majority of each decode step while waiting on memory transfers[51]. Speculative decoding exploits this idle compute by proposing γ candidate tokens cheaply and verifying all $\gamma + 1$ positions in a **single parallel forward pass** of the target model, thereby reading the target model's weights from memory only once[51]. Assuming each proposed token is accepted with probability α and a single draft step costs a fraction c of a target forward pass, the expected number of generated tokens per target step and the resulting net speedup in the memory-bound regime are governed by[51]:

$$E[\text{tokens per step}] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}, \qquad \text{Speedup}_{\text{memory-bound}} = \frac{1 - \alpha^{\gamma+1}}{(1 - \alpha)(1 + \gamma c)}$$

At $\alpha = 0.8$ and $\gamma = 4$, the target model emits an average of **3.36 tokens per weight read** while guaranteeing a mathematically identical output distribution via modified rejection sampling[51]. On 16 GB to 24 GB accelerators (T4, L4, TPU v5e) where loading a separate draft model competes with the KV cache for scarce memory[52], practitioners should enable **Prompt Lookup (N-Gram) Speculative Decoding** (--speculative-config '{"method": "ngram", "num_speculative_tokens": 5, "prompt_lookup_max": 5, "prompt_lookup_min": 2}')[52]. This technique requires **zero additional VRAM** ($c \approx 0$) by matching 2-to-5 token n-grams from the current generation suffix against earlier tokens in the prompt and copying the subsequent span[52]. For input-echoing workloads—such as Retrieval-Augmented Generation (RAG), document summarization, JSON extraction, and code refactoring—n-gram lookup provides a 1.8× to 2.5× speedup at zero memory cost[51].

For open-ended generation, mathematical reasoning, or chat on high-capacity accelerators (G4, A100, TPU v6e) where outputs do not repeat the prompt, n-gram lookup yields low acceptance rates[52]. In these scenarios, **Multi-Token Prediction (--method mtp)** on architectures with built-in MTP heads (such as Qwen3.5/3.6 or DeepSeek) achieves **91% to 100% token acceptance rates** on Blackwell SM120[32], while lightweight **EAGLE-3 (--method eagle3)** or Arctic MLP speculators can be quantized to FP8 to minimize draft latency c [52]. On TPUs, vllm-jax and vLLM-TorchTPU natively support speculative decoding, and recent vllm-jax releases optimize the **Batched RPA** kernel to categorize speculative verification as a decode operation rather than a mixed prefill, eliminating padding overhead[17].

Crucially, speculative decoding is strictly a low-concurrency optimization[51]. Once concurrent batch size increases and the accelerator becomes compute-bound, verifying $\gamma + 1$ tokens

costs roughly $\gamma + 1$ times the compute of a single token[51]:

$$\text{Speedup}_{\text{compute-bound}} = \frac{E[\text{tokens per step}]}{(\gamma + 1) + \gamma c}$$

Under high QPS on 70B models, unthrottled speculative decoding causes a **1.4× slowdown on ShareGPT** and a **1.8× slowdown on CNN/DailyMail**, dropping effective throughput to 0.56×–0.71× of the non-speculative baseline[51]. To prevent high-batch degradation in vLLM, configurations must keep num_speculative_tokens between **2 and 4** (or up to 5 for structured code/JSON)[51], set --speculative-disable-by-batch-size 4 (or 8) so the scheduler automatically disables speculation when active concurrency crosses into the compute-bound regime[51], and leave --disable-logprobs-during-spec-decoding enabled (True by default) to skip redundant log-probability calculations across proposal, target, and rejection steps[54].

Memory Subsystem, Compiler Flags, and Colab I/O Tuning

Because Google Colab instances operate inside virtualized container environments with network-attached storage and session timeouts, runtime initialization and memory scheduling settings significantly impact sustained token generation rates. Enabling --enable-chunked-prefill splits long incoming prompts into bounded token chunks (--max-num-batched-tokens), piggybacking prefill compute onto active decode batches so a single long prompt does not stall inter-token latency (TPOT) for concurrent streams or trigger activation OOM spikes[25]. Pairing this with --enable-prefix-caching retains KV cache blocks for shared system prompts or multi-turn histories in memory, which is supported across both CUDA GPUs and hybrid linear-attention models on TPUs (mamba_cache_mode="align")[17].

At the scheduler and loss-computation level, increasing --num-scheduler-steps (e.g., to 4 or 8) in vLLM allows the engine to execute multiple consecutive forward passes on the accelerator before returning control to the Python scheduler, hiding CPU overhead on fast GPUs like the G4 and A100[54]. For training or fine-tuning runs on TPUs via MaxText, setting num_vocab_tiling=8 tiles the cross-entropy loss logit projection across the vocabulary dimension, substantially reducing peak HBM usage so larger batch sizes fit in memory[36]. Finally, because loading multi-gigabyte checkpoints in Colab frequently bottlenecks on virtual disk I/O, passing --load-format fastsafetensors in vLLM uses direct zero-copy memory transfers to load 27B quantized checkpoints in **1.69 to 3.04 seconds**[27]. To avoid paying a 5-to-10 minute XLA compilation or FlashInfer JIT compilation penalty every time a Colab runtime disconnects, mounting Google Drive or a Google Cloud Storage (GCS) bucket and pointing **FlashInfer's JIT cache (export FLASHINFER_CACHE_DIR=/content/drive/MyDrive/flashinfer_cache)** on GPUs[40] and **TorchTPU / OpenXLA's Tier-3 Persistent Compilation Cache** on TPUs[47] to persistent storage ensures immediate warm-start execution across notebook sessions.

Synthesis of Optimal Configurations for Google Colab Workloads

The following deployment matrix consolidates the corrected precision formats, specialized

kernel backends, and runtime execution flags required to achieve peak token generation throughput across every accelerator available on the Google AI Colab plan.

Colab Hardware Target	Workload Regime	Optimal Precision & Checkpoint Format	Primary Engine & Kernel Backend	Mandatory Runtime Flags & Environment Settings
G4 GPU (RTX PRO 6000 Blackwell, 96 GB)	**Low-Batch Decode (** $B =$ **)**	**NVFP4 / MXFP4 (W4A4)** or **AWQ/GPTQ (W4A16)** + **FP8 / NVFP4 KV Cache**[2]	vLLM / SGLang with **FlashInfer (cu13 + compute_120f)** & **CUTLASS**[22]	pip install flashinfer-python[cu13]; VLLM_NVFP4_GEMM_BACKEND=cutlass; VLLM_USE_FLASHINFER_MOE_FP4=1 (if MoE); --compilation-config '{"cudagraph_mode":"PIECEWISE"}'; --speculative-config '{"method":"mtp","num_speculative_tokens":3}'[22]
G4 GPU (RTX PRO 6000 Blackwell, 96 GB)	**High-Batch Serving (** $B \geq$ **)**	**NVFP4 (W4A4)** or **Static FP8 (W8A8 E4M3)** + **FP8 KV Cache**[2]	vLLM / TensorRT-LLM with **5th-Gen Tensor Core FP4/FP8 GEMM**[2]	--kv-cache-dtype fp8; --enable-chunked-prefill; --enable-prefix-caching; --gpu-memory-utilization 0.95; optional **4× 24 GB MIG**

				partitioning for multi-worker data parallelism[2]
A100 (40 GB PCIe / 80 GB SXM)	**Low-Batch Decode (** $B =$ **)**	**W4A16 (AWQ / GPTQ)** or **W8A16 (FP8 Checkpoint)** + **FP16/BF16 KV Cache**[3]	vLLM with **awq_marlin / gptq_marlin / fp8 (Marlin)** + **FlashInfer**[3]	--quantization awq_marlin; VLLM_ATTENTION_BACKEND=FLASHINFER; --speculative-config (EAGLE-3 or N-Gram with --speculative-disable-by-batch-size 4); avoid fp8_e5m2 KV cache unless memory-capacity bound[3]
A100 (40 GB PCIe / 80 GB SXM)	**High-Batch Serving (** $B \geq$ **)**	**W8A8 SmoothQuant INT8** or **W4A16 Marlin** (with 2:4 Sparsity if available)[3]	TensorRT-LLM or vLLM (**INT8 Tensor Cores @ 624 TOPS**)[3]	--enable-chunked-prefill; --enable-prefix-caching; torch.backends.cuda.matmul.allow_tf32 = True; use kv_cache_dtype="fp8_e5m2" only with piecewise/eager-break patch if 100K+ context requires 8-bit KV[3]
L4 GPU (Ada	**Low-Batch**	**W4A16 (AWQ /**	vLLM with	--quantization

Lovelace, 24 GB)	**Decode (** $B =$ **)**	**GPTQ) + FP8 KV Cache**[4]	**awq_marlin / gptq_marlin + FlashInfer**[4]	awq_marlin; --kv-cache-dtype fp8; VLLM_ATTENTION_BACKEND=FLASHINFER; --speculative-config '{"method":"ngram","num_speculative_tokens":5}' for RAG/code[4]
L4 GPU (Ada Lovelace, 24 GB)	**High-Batch Serving (** $B \geq$ **)**	**Static FP8 (W8A8 E4M3) + FP8 KV Cache**[4]	vLLM / TensorRT-LLM (**Ada FP8 Tensor Cores @ 242 TFLOPS**)[4]	--quantization fp8 (prefer static activation scales); --kv-cache-dtype fp8; --enable-chunked-prefill; --gpu-memory-utilization 0.92[4]
Tesla T4 (Turing, 16 GB)	**All Inference Regimes (** $B =$ **)**	**INT4 (EXL2 3.0–4.0 bpw, AWQ, or GGUF Q4_K_M)** or **W8A16 FP8**[5]	ExLlamaV2 (fastest $B =$), vLLM (**SM75 Marlin**), or llama.cpp (**CUDA MMQ**)[5]	Strictly dtype="float16" (never bfloat16); in llama.cpp pass -DGGML_CUDA_FORCE_MMQ=ON -fa -ctk q8_0 -ctv q8_0; in vLLM use --quantization awq_marlin and N-Gram prompt lookup[5]

TPU v6e (Trillium, 32 GB HBM)	**Low-Batch Decode (** $B =$ **)**	**W4A16 (compressed-tensors / AWQ)** or **FP8 (W8A8 E4M3)** + **FP8 KV Cache**[1]	vllm-jax (**gmm_v2 W4A16 + Stacked RPA v3**) or TorchTPU (torch.compile)[17]	--kv-cache-dtype fp8; select **Batched/Stacked RPA v3** attention backend; align Pallas/Tokamax subchannel blocks to multiples of **256** ($\geq$); buffer tokens on-chip before host readback[1]
TPU v6e (Trillium, 32 GB HBM)	**High-Batch Serving (** $B \geq$ **)**	**W8A8 INT8 (AQT / compressed-tensors)** or **W8A8 FP8** + **FP8 KV Cache**[1]	vllm-jax or JetStream + MaxText (**1,836 TOPs INT8 / 918 TFLOPs FP8 MXU**)[8]	Use **W8A8 INT8** to hit the 1,836 TOPs ceiling; --enable-chunked-prefill; --enable-prefix-caching; offload embeddings/collectives to **SparseCore** streams; enable Tier-3 persistent XLA cache[8]
TPU v5e (16 GB HBM)	**Low-Batch (** $B =$ **) vs. Batch (** $B \geq$ **)**	$B =$ **: W4A16 + 4-Bit Embed Grid**; $B \geq$ **: W8A8 INT8** + **INT8 KV Cache**[6]	vllm-jax (gmm_v2) or TorchTPU (torch.compile(backend="tpu"))[17]	Use a **Static Ring KV Cache** with **Bounded Dynamism**; keep embedding table in 4-bit with BF16 scale (frees 3.4

				GB HBM, +1.55× speed); accumulate tokens on-chip to avoid the 32–37 tok/s host-streaming cap[18]

Works cited

1. TPU v6e Precision Formats.md
2. Optimal Precision Formats for G4 GPU.md
3. Optimal Precision Formats for A100.md
4. L4 GPU Precision Formats on Colab.md
5. Optimal Precision Formats for T4.md
6. Optimal TPU v5e Precision Formats.md
7. Google Cloud TPU v6e (Trillium) | AI Hardware Directory, https://www.madebyagents.com/hardware/google-cloud-tpu-v6e-trillium
8. TPU v6e - Google Cloud Documentation, https://docs.cloud.google.com/tpu/docs/v6e
9. Google TPU Architecture Versions Explained: From v1 to the Eighth, https://medium.com/womenintechnology/google-cloud-tpu-architecture-versions-explained-from-v1-to-the-eighth-generation-3f2371962154
10. TPU machines in accelerator-optimized machine family, https://docs.cloud.google.com/compute/docs/tpus/tpu-machines
11. TPUs vs GPUs: When to Choose What for AI/ML Workloads - Cast AI, https://cast.ai/blog/tpu-vs-gpu/
12. Feature Matrix - vLLM TPU, https://docs.vllm.ai/projects/tpu/en/v0.31.0/features/
13. TPU v6e vs A100 80GB×2 for Gemma 4 31B on vLLM - Implicit None, https://implicit-none.com/en/vllm-tpu-gpu-benchmark-2026-april/
14. Perform multihost inference using Pathways | AI Hypercomputer, https://docs.cloud.google.com/ai-hypercomputer/docs/workloads/pathways-on-cloud/multihost-inference
15. TPU generations, precision and profiling | JAX Pathways, https://www.tahabouhsine.com/jaxpathways/tpu-performance.html
16. Google Trillium TPU (v6e) introduction : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1gnru7c/google_trillium_tpu_v6e_introduction/
17. Releases · vllm-project/tpu-inference - GitHub, https://github.com/vllm-project/tpu-inference/releases
18. PyTorch on TPU: Hands On With Colab - DEV Community, https://dev.to/gde/pytorch-on-tpu-hands-on-with-colab-5b5g
19. NVIDIA RTX Pro 6000 96GB GDDR7 Blackwell Server Graphic Card, https://www.serverbasket.com/shop/nvidia-rtx-pro-6000-blackwell-server-edition-g

pu/
20. NVIDIA RTX PRO 6000 Blackwell Server Edition - Inferbase, https://inferbase.ai/gpu-catalog/gpu/nvidia-rtx-pro-6000-blackwell-server-edition
21. NVIDIA RTX PRO 6000 Server Edition: Specs & Rental | Akash, https://akash.network/pricing/gpus/rtx-pro-6000-server/
22. SM120 (RTX Blackwell) NVFP4 MoE: CUTLASS Grouped GEMM, https://www.reddit.com/r/LocalLLaMA/comments/1roqkw4/sm120_rtx_blackwell_nvfp4_moe_cutlass_grouped/
23. [Bug] NVFP4 mm_fp4 GEMM broken on SM120 (RTX PRO ... - GitHub, https://github.com/flashinfer-ai/flashinfer/issues/2577
24. and Source-Level Auditof INT8 Availability on NVIDIA Blackwell Ultra, https://arxiv.org/html/2608.11693v1
25. [Bug]: MiniMax-M3 fp8 KV cache on SM80 (A100/A800 ... - GitHub, https://github.com/vllm-project/vllm/issues/50881
26. [Bug]: NVIDIA A100 actually supports fp8_e5m2 quantization. #3965, https://github.com/vllm-project/vllm/issues/3965
27. marlin - vLLM, https://vllm.website.cncfstack.com/api/vllm/model_executor/kernels/linear/scaled_mm/marlin/
28. vLLM GPU Compatibility Matrix (Compute Capability 7.0 ... - Speediyo, https://www.speediyo.com/ai-infra/vllm-gpu-compute-capability-matrix
29. FP8 W8A8 - vLLM docs, https://docs.vllm.ai/en/stable/features/quantization/llm_compressor/fp8/
30. Advanced Configuration — NVIDIA NIM for LLM and VLM, https://docs.nvidia.com/nim/large-language-models/2.0.13/reference/advanced-configuration.html
31. Release Highlights - FlashInfer, https://flashinfer.ai/releases/
32. nvfp4 KV cache on SM120 — flashinfer ships the kernels, vLLM isn't, https://github.com/vllm-project/vllm/issues/49011
33. The Complete Guide to LLM Quantization with vLLM - Jarvislabs.ai, https://jarvislabs.ai/blog/vllm-quantization-complete-guide-benchmarks
34. Google TPU Trillium v6 vs NVIDIA B200: LLM Inference Cost and, https://www.spheron.network/blog/google-tpu-trillium-v6-vs-nvidia-b200-llm-inference/
35. HW Fit rates AWQ/GPTQ/FP8 models as good fits on Pascal and, https://github.com/odysseus-dev/odysseus/issues/6402
36. Inside the optimization of FP8 training on Ironwood, https://discuss.google.dev/t/inside-the-optimization-of-fp8-training-on-ironwood/336681
37. GPU machine types | Compute Engine - Google Cloud Documentation, https://docs.cloud.google.com/compute/docs/gpus
38. Google Colab, https://colab.research.google.com/
39. FlashInfer: Kernel Library for LLM Serving - GitHub, https://github.com/flashinfer-ai/flashinfer
40. Deploy FlashInfer on GPU Cloud: LLM Inference Kernels for vLLM,

https://www.spheron.network/blog/deploy-flashinfer-gpu-cloud-llm-inference-kernels/
41. FP8 W8A8 - vLLM Documentation, https://docs.vllm.ai/en/v0.20.0/features/quantization/fp8/
42. ThinkSystem NVIDIA RTX PRO 6000 Blackwell Server Edition PCIe, https://lenovopress.lenovo.com/lp2263-thinksystem-nvidia-rtx-pro-6000-blackwell-server-edition-pcie-gen5-gpu
43. vLLM brings FP8 inference to the open source community, https://developers.redhat.com/articles/2024/07/15/vllm-brings-fp8-inference-open-source-community
44. TensorRT-LLM FP8 on A100 - Quantization Setup Guide (2026), https://blog.easecloud.io/ai-cloud/optimize-gpus-faster-with-tensorrt-llm/
45. Does a Second GPU Increase Ollama's Context Window? (Quadro, https://dev.to/sikamikanikobg/does-a-second-gpu-increase-ollamas-context-window-quadro-p2000-rtx-3090-tested-5hbh
46. [PRD] Bounded Dynamic Shape Design for `PyTorch` & `PyTorch/XLA`, https://github.com/pytorch/xla/issues/3884
47. Introducing TorchTPU - PyTorch, https://pytorch.org/blog/introducing-torchtpu/
48. Accelerating Workloads (Part 2) in the Google Cloud Ecosystem, https://wearecommunity.io/communities/tpucommunity/articles/8898
49. A Hybrid Eager/Compiled Architecture for Native PyTorch Execution, https://arxiv.org/html/2610.24161v2
50. GitHub - vllm-project/tpu-inference: Moving to vllm-project/vllm-jax, https://github.com/vllm-project/tpu-inference
51. Speculative Decoding Made My vLLM Server Slower, https://dev.to/ji_ai/speculative-decoding-made-my-vllm-server-slower-the-acceptance-math-49if
52. Speculative Decoding in vLLM: A Practical Guide to Faster LLM, https://acingai.com/articles/speculative-decoding-vllm
53. How Speculative Decoding Boosts vLLM Performance by up to 2.8x, https://vllm.ai/blog/2024-10-17-spec-decode
54. Engine Arguments - vLLM Documentation, https://docs.vllm.ai/en/v0.6.0/models/engine_args.html
55. Speculative Decoding - vLLM Documentation, https://docs.vllm.ai/en/stable/examples/features/speculative_decoding/
56. Fastest Speculative Decoding in vLLM with Arctic Inference and, https://www.snowflake.com/en/blog/engineering/fast-speculative-decoding-vllm-arctic/
57. Recommended Models and Features - vLLM TPU, https://docs.vllm.ai/projects/tpu/en/v0.12.0/recommended_models_features/
58. vLLM Performance Tuning: The Ultimate Guide to xPU Inference, https://cloud.google.com/blog/topics/developers-practitioners/vllm-performance-tuning-the-ultimate-guide-to-xpu-inference-configuration
59. Serve LLMs using multi-host TPUs on GKE with JetStream and, https://docs.cloud.google.com/kubernetes-engine/docs/tutorials/serve-multihost-tp

u-jetstream